

Convertir un programme en C en C++

1

```
#include <stdio.h>
#define N 40
void sum(int *p, int n, int d[])
{
    int i;
    *p = 0 ;
    for (i=0 ; i<n;++i) *p =*p + d[i];
}

int main ( )
{
    int i;
    int accum = 0;
    int data [N];

    for (i=0; i<N; ++i) data [i] = i ;
    sum(&accum, N, data ) ;
    printf("sum i s %d\n",accum);
    return (0);
}
```

□

Exercices en C++

2

Reprendre les codes des Slides 19, 30, 33, 35, 40, 48, 50, 52, 63 et 69, les programmer et les exécuter.

□

3 Reprendre le code du Slide 39 et le programmer. Essayer de trouver la bibliothèque manquante. ☐

4 Réaliser un programme simple additionnant deux nombres entiers en C++. On se contentera juste de faire un programme main(). On demandera à l'utilisateur d'entrer les entiers et le programme réalisera l'opération. ☐

5 Modifier le programme pour tenir compte des autres opérations : soustraction, multiplication et division. Le programme demandera à l'utilisateur quelle opération il désire réaliser (+,-,*,/). On donnera le résultat sous la forme $2 + 3 = 5$. L'utilisateur pourra refaire des opérations plusieurs fois. ☐

6 Une des solutions possibles est de finir le programme de la façon suivante :

```
do
{
.
.
.
char rep ;
do
{
cout << "Voulez-vous recommencer (o /n) ? " << endl ;
cin >> rep;
} while ((rep != 'o' ) && (rep != 'n'));
} while (rep=='o');
return 0;
}
```

Est-ce que cette fin de programme compile et fonctionne? Si non, pourquoi? ☐

7

Dans l'utilisation de `cout` et `cin`, on a la possibilité de modifier la façon dont les éléments sont lus ou écrits dans le flot : — `dec` lecture/écriture d'un entier en décimal — `oct` lecture/écriture d'un entier en octal — `hex` lecture/écriture d'un entier en hexadécimal — `endl` insère un saut de ligne et vide les tampons — `flush` vide les tampons après écriture — `boolalpha` permet d'écrire la valeur d'un booléen — `setw(int n)` affichage de `n` caractères (utiliser la librairie de l'exercice précédent) — `setprecision(int n)` affichage de la valeur avec `n` chiffres avec éventuellement un arrondi de la valeur (utiliser la librairie de l'exercice précédent) — `setfill(char)` définit le caractère de remplissage (utiliser la librairie de l'exercice précédent)

Programmer un code C++ qui permet la mise en forme suivante :

```
|*****4 d2|  
|*12.35|
```

On déclarera deux variables :

```
int i=1234;  
float p=12.3456;
```

Déclarer une variable booléenne initialisée à vrai. Afficher la valeur de cette variable. □